An Open-Source Modular Benchmark for Diffusion-Based Motion Planning in Closed-Loop Autonomous Driving

Anonymous Submission

***Abstract*— Diffusion-based motion planners have achieved state-of-the-art results on benchmarks such as nuPlan, yet their evaluation within closed-loop production autonomous driving stacks remains largely unexplored. Existing evaluations abstract away ROS 2 communication latency and real-time scheduling constraints, while monolithic ONNX deployment freezes all solver parameters at export time. We present an open-source modular benchmark that addresses both gaps: using ONNX GraphSurgeon, we decompose a monolithic 18,398-node diffusion planner into three independently executable modules and reimplement the DPM-Solver++ denoising loop in native C++. Integrated as a ROS 2 node within Autoware, the open-source AD stack deployed on real vehicles worldwide, the system enables runtime-configurable solver parameters without model recompilation and per-step observability of the denoising process, breaking the black box of monolithic deployment. Unlike evaluations in standalone simulators such as CARLA, our benchmark operates within a production-grade stack and is validated through AWSIM closed-loop simulation. Through systematic comparison of DPM-Solver++ (first- and second-order) and DDIM across six step-count configurations ($N \in \{3, 5, 7, 10, 15, 20\}$), we show that encoder caching yields a $3.2\times$ latency reduction, and that second-order solving reduces FDE by 41% at $N{=}3$ compared to first-order. The complete codebase will be released as open-source, providing a direct path from simulation benchmarks to real-vehicle deployment. Project page: `https://anonymous.4open.science/r/modular-diffusion-planner-3DDA/`.**

I. Introduction

Diffusion models have emerged as a powerful paradigm for motion planning in autonomous driving (AD) [1]–[4]. By formulating trajectory generation as iterative denoising from a learned data distribution, diffusion-based planners can jointly model multi-agent futures and produce temporally consistent plans. Recent works such as Diffusion Planner [1], DiffusionDrive [2], and Latent Planner (LAP) [4] report state-of-the-art results on the nuPlan benchmark [5], [6]. However, translating these benchmark results into deployable systems remains challenging, due to two largely unaddressed issues.

Nearly all existing diffusion planners are evaluated in *pseudo-closed-loop* settings that replay recorded scenarios via Python-based simulators or nuPlan's reactive agents [5]. While useful for model comparison, these settings abstract away the latency constraints of production AD stacks: ROS 2 topic communication overhead [7], [8], sensor preprocessing pipelines, and real-time scheduling on shared vehicle compute. Consequently, the gap between "benchmark performance" and "vehicle-deployable performance" remains unmeasured. As shown in §V-C, the monolithic diffusion planner studied in this work requires over 300 ms per planning cycle on CPU, far exceeding the typical 100 ms budget of Autoware-class planning nodes [7].

In addition, current deployment pipelines export the entire multi-step denoising process, including the noise schedule, solver logic, and iterative DiT blocks, into a single monolithic ONNX graph. In the case of Diffusion Planner [1], this produces 18,398 nodes, which are then compiled into a TensorRT engine. This design freezes all solver parameters at export time: step count ($N{=}10$), solver order, and noise schedule cannot be changed without re-exporting and recompiling the model. This prevents researchers from studying fundamental questions such as: *How many denoising steps are truly needed under real-time constraints? Does second-order solving provide meaningful gains over first-order in closed-loop driving? Can anytime planning [9] be realized by adaptively truncating diffusion steps?*

Both problems trace to a single design choice: embedding the iterative denoising loop *inside* the computational graph. Our key idea is to move the loop *outside*, replacing the monolithic model with independently executable modules orchestrated by an external C++ solver. Specifically, we use ONNX GraphSurgeon [10] to decompose the monolithic graph into three modules: a context encoder (3,417 nodes, 18 MB), executed once per planning cycle and cached; a DiT core (1,237 nodes, 28 MB), invoked N times where N is configurable at runtime; and a turn indicator classifier (7 nodes, 5.6 KB) for post-hoc signal prediction.

The DPM-Solver++ [11] loop is reimplemented in native C++ with runtime-configurable step count ($N \in [1, 50]$), solver order, and VP noise schedule [12] parameters. The system is integrated as a drop-in ROS 2 node within the Autoware [13] stack and validated in AWSIM [14], the open-source simulator designed for closed-loop evaluation within the Autoware ecosystem. The modular design creates a pluggable solver interface: researchers can substitute alternative denoising strategies (DDIM [15], higher-order DPM-Solver variants, or novel schedulers) without modifying the neural network weights or retraining, and immediately evaluate them under realistic closed-loop conditions.

Our contributions are as follows.

First, we propose an open-source modular benchmark that decomposes a monolithic ONNX diffusion planner into independently executable modules via GraphSurgeon, enabling plug-and-play solver evaluation in the Autoware

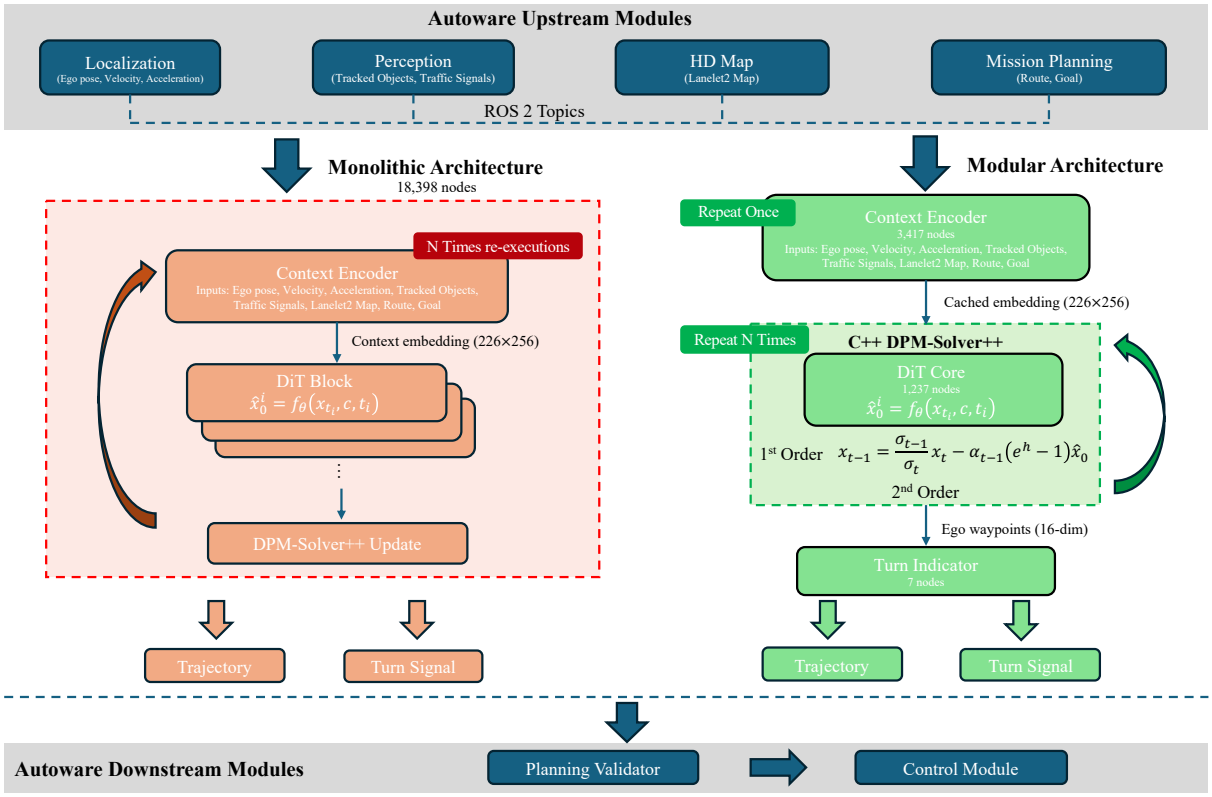

Fig. 1. **(a)** Monolithic ONNX graph (18,398 nodes) compiled to TensorRT. **(b)** Modular decomposition: three independently executable modules with the DPM-Solver++ loop in C++; the context encoder runs once and is cached across all denoising steps.

+ AWSIM production AD stack under true closed-loop conditions.

Second, we provide a native C++ DPM-Solver++ implementation with runtime-configurable inference parameters, demonstrating that solver behavior, including step count, order, and noise schedule, can be studied without model recompilation, a capability absent from all existing diffusion planner deployments.

Third, through systematic solver comparison across DPM-Solver++ (first- and second-order) and DDIM over 50 AWSIM frames with $N \in \{3, 5, 7, 10, 15, 20\}$, we show that second-order correction reduces FDE by 41% at N=3 and that proper timestep scheduling improves quality by 7× over naive early stopping.

Finally, we validate numerical equivalence with the monolithic model (max error $< 10^{-5}$), demonstrate encoder caching (90.9% computation reduction, 3.2× latency improvement), and confirm successful urban driving in AWSIM. The complete codebase will be released as open-source.

II. Related Work

A. Diffusion Models for Motion Planning

Diffusion models [12], [16] have been increasingly applied to autonomous driving. Diffuser [17] pioneered diffusion-based trajectory planning for robotics; Diffusion Policy [18] demonstrated visuomotor control via action diffusion. In the AD domain, Diffusion Planner [1] jointly models ego and neighbor trajectories using a DiT architecture [19] with a variance-preserving (VP) noise schedule, achieving state-of-the-art on the nuPlan benchmark [5]. DiffusionDrive [2] introduces truncated diffusion with anchored initialization, reducing steps to 2. BridgeDrive [3] formulates planning as a diffusion bridge; LAP [4] uses fine-grained latent features for fast inference; Diffusion-ES [20] combines diffusion with gradient-free optimization; and DriveLaW [21] unifies planning with video generation. Despite these algorithmic advances, *all* of these methods are evaluated on dataset replay or Python-based simulators, not within production AD stacks under real-time constraints.

B. Closed-Loop Evaluation in Production Stacks

The nuPlan benchmark [5] provides a "closed-loop" evaluation by replaying scenarios with reactive agents. However, this is a *pseudo-closed-loop*: the planner runs in a Python process without the middleware, communication, and scheduling constraints present in production systems. In real-world AD deployments, ROS 2 serves as the standard middleware framework, providing the publish-subscribe communication infrastructure between perception, planning, and control modules. Dauner et al. [6] showed that even simple

rule-based planners can achieve competitive nuPlan scores, questioning whether benchmark rankings translate to real-world performance. Betz et al. [7] further demonstrated that ROS 2-based AD stacks introduce significant communication latency that cascades through the full pipeline, fundamentally altering the operating conditions. Our work bridges this gap by benchmarking within Autoware [13], a fully open-source ROS 2-based AD stack with real-vehicle deployments by organizations worldwide. Unlike proprietary platforms such as Baidu Apollo [22], Autoware's open-source nature enables fully reproducible research with a direct path from simulation to on-vehicle deployment.

C. Model Deployment and Partitioning

Deploying DNN models on vehicle platforms typically involves ONNX export followed by hardware-specific compilation [23], [24]. Guo et al. [25] propose EASTER, which learns optimal split points for transformers across edge devices. Nair et al. [26] introduce SecureFrameNet for partitioning ONNX models in secure deployment scenarios. Jajal et al. [27] analyze ONNX converter interoperability failures, identifying systematic issues across deep learning frameworks. However, these approaches target general-purpose models and do not exploit the *iterative* structure of multi-step diffusion models, where the same denoising subnetwork is invoked repeatedly within a solver loop.

D. Diffusion Inference Acceleration

A complementary line of work focuses on accelerating the diffusion sampling process itself. DPM-Solver++ [11] is a high-order ODE solver for diffusion models that reduces the required denoising steps from hundreds as in DDPM [16] to 10–20 while maintaining sample quality. DDIM [15] enables deterministic sampling via an implicit non-Markovian process. Step-reduction approaches such as truncated diffusion [2] reduce steps further but require model-specific training. More recently, flow matching and consistency models have enabled one- or two-step generation, though their application to production AD planning remains limited. Our work is *orthogonal and complementary*. Rather than proposing a new solver, we provide an open-source platform where *any* solver can be plugged in and evaluated under closed-loop driving conditions.

III. System Architecture

A. Diffusion Planner in the Autoware Stack

Fig. 1 provides an overview of the system architecture. Within the Autoware stack, the diffusion planner operates as the planning module, receiving perception outputs, including tracked objects, HD vector map, route, and traffic signals, and producing a trajectory that the downstream control module executes. The Diffusion Planner [1] itself consists of three functional components:

Context Encoder. The encoder processes the scene context $\mathcal{I} = \{s_{\text{ego}}, s_{\text{nbr}}, s_{\text{lane}}, s_{\text{route}}, s_{\text{tl}}, s_{\text{goal}}\}$, comprising ego vehicle history, neighbor trajectories, lane geometry, route information, traffic signals, and goal pose. These heterogeneous inputs are fused through a mixer-attention architecture into a context embedding $\mathbf{c} \in \mathbb{R}^{L\times d}$ (L=226, d=256).

Diffusion Transformer (DiT). The DiT backbone takes as input the noisy joint trajectory $\mathbf{x}_t \in \mathbb{R}^{A\times T\times 4}$ (A=33 agents, T=81 timesteps, state $(x, y, \cos\theta, \sin\theta)$), the context embedding $\mathbf{c}$, and a continuous timestep $t \in [0, 1]$. It outputs a denoised trajectory estimate $\hat{\mathbf{x}}_0 \in \mathbb{R}^{A\times(T-1)\times 4}$.

Turn Indicator Predictor. A linear classifier maps the context embedding $\mathbf{c}$ and sampled ego waypoints to a 4-class turn indicator prediction.

B. VP Noise Schedule and DPM-Solver++

Diffusion models [12], [16] generate data by learning to reverse a noise-adding forward process. Given a clean trajectory $\mathbf{x}_0$, the forward process produces a noisy version $\mathbf{x}_t = \alpha(t)\mathbf{x}_0 + \sigma(t)\boldsymbol{\epsilon}$ at continuous time $t \in [0, 1]$, where $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$. The VP noise schedule [12] parameterizes the signal and noise magnitudes as:

$$\log \alpha(t) = -\tfrac{1}{4}t^2(\beta_1 - \beta_0) - \tfrac{1}{2}t\beta_0, \tag{1}$$

with $\sigma(t) = \sqrt{1-\alpha(t)^2}$. The log signal-to-noise ratio $\lambda(t) = \log[\alpha(t)/\sigma(t)]$ decreases monotonically from clean data (t≈0) to near-pure noise (t≈1). The Diffusion Planner uses $\beta_0 = 0.1$ and $\beta_1 = 20.0$.

At inference, the DiT predicts the clean trajectory $\hat{\mathbf{x}}_0$ from the noisy input $\mathbf{x}_t$ and context $\mathbf{c}$, and a solver iteratively denoises from t=1 toward t=0. DPM-Solver++ [11] is a dedicated high-order ODE solver for this reverse process. The first-order update from timestep t_s to t is:

$$\mathbf{x}_t = \frac{\sigma_t}{\sigma_s}\mathbf{x}_s + \alpha_t(1 - e^{-h})\hat{\mathbf{x}}_0, \tag{2}$$

where $h = \lambda_t - \lambda_s$ is the logSNR difference between the two timesteps. When a previous prediction is available, the second-order update refines the estimate by incorporating a finite-difference correction:

$$\mathbf{x}_t = \frac{\sigma_t}{\sigma_s}\mathbf{x}_s + \alpha_t(1 - e^{-h})\hat{\mathbf{x}}_0 + \frac{1}{2}\alpha_t(1 - e^{-h})D_1, \tag{3}$$

where $D_1 = (\hat{\mathbf{x}}_0^{(i)} - \hat{\mathbf{x}}_0^{(i-1)})/r$ is the finite-difference derivative with $r = h_{i-1}/h_i$. This correction leverages consecutive predictions to better approximate the denoising trajectory, achieving higher accuracy with the same number of neural network evaluations.

C. Monolithic Deployment

In the original Autoware integration, the entire diffusion process, including 11 DiT steps, the VP schedule, and the DPM-Solver++ logic, is *unrolled* into a single ONNX graph at export time. This produces a model with **18,398 nodes**, compared to 1,237 for a single DiT step, that is compiled into a ∼68 MB TensorRT engine. While this enables aggressive kernel fusion, it freezes all solver parameters and forces the

context encoder to re-execute at every denoising step. The total inference cost per planning cycle is:

$$T_{\text{mono}} = (N{+}1) \cdot T_{\text{enc}} + (N{+}1) \cdot T_{\text{dit}} + N \cdot T_{\text{sol}}, \quad (4)$$

where T_{enc}, T_{dit}, and T_{sol} denote the latency of the context encoder, a single DiT step, and the solver update, respectively. Since $T_{\text{enc}} \gg T_{\text{dit}}$ as quantified in §V-C, the redundant $(N{+}1)$ encoder invocations dominate the total cost. Fig. 1(a) illustrates this monolithic design; our proposed modular alternative is detailed in §IV and shown in Fig. 1(b).

IV. Methodology

A. Graph Decomposition via ONNX GraphSurgeon

As shown in Fig. 1(a), the monolithic ONNX graph contains a *repeated substructure*: the same DiT block appears 11 times with shared weights but different timestep inputs. We exploit this structure to decompose the graph into three independently executable modules (Fig. 1(b)).

Step 1: Subgraph Identification. Using ONNX GraphSurgeon [10], we analyze the computational graph to identify: (a) the context encoder subgraph, whose output feeds into all DiT blocks but receives no feedback from them; (b) a single DiT block, identified by weight sharing across the 11 unrolled copies; and (c) the turn indicator linear layer at the graph's output.

Step 2: Subgraph Extraction. Each subgraph is extracted into an independent ONNX model with explicitly defined inputs and outputs. The encoder's output node (context embedding $\mathbf{c}$) becomes both the encoder model's output and the DiT model's input. The timestep, which was a constant in each unrolled copy, becomes a dynamic input to the extracted DiT model. Concretely, we use GraphSurgeon to create new input `Variable` nodes at each subgraph boundary, specifically the encoder's final LayerNormalization output and the DiT's output projection, rewire all downstream consumer nodes, and invoke automatic graph cleanup to prune the now-unreachable encoder and duplicate DiT nodes. The turn indicator is reconstructed as a minimal 7-node graph from the extracted linear layer weights.

Step 3: Validation. We verify numerical equivalence by running both the original monolithic model and the decomposed pipeline on identical inputs, confirming a maximum element-wise error below 10^{-5}; see §V-A.

Table I summarizes the decomposition.

Unlike the monolithic TensorRT engine, the decomposed ONNX modules can be executed via ONNX Runtime [23] on any supported backend, including CPU, CUDA, and other accelerators, without recompilation.

B. C++ DPM-Solver++ Implementation

With the DiT core extracted as a single-step model, we reimplement the DPM-Solver++ denoising loop in native C++. This choice is driven by the Autoware ecosystem: Autoware nodes are implemented in C++, the ONNX Runtime C++ API avoids Python interpreter overhead and GIL constraints, and deterministic memory management is essential for meeting the real-time scheduling requirements of production AD stacks. Algorithm 1 presents the full procedure.

TABLE I

Monolithic vs. Modular Architecture.

	Monolithic	**Modular (ours)**
ONNX nodes	18,398	4,661[†]
Model size	47 MB	46 MB
Enc. calls / cycle	$N{+}1$	1 (cached)
Runtime backend	TensorRT (GPU)	ONNX Runtime (CPU/GPU)
Solver config	Frozen at export	Runtime-configurable

[†]Encoder (3,417) + DiT core (1,237) + Turn indicator (7).

Algorithm 1 Modular Diffusion Planning with DPM-Solver++

Require: Scene inputs $\mathcal{I}$, steps N, order $p \in \{1, 2\}$

1: $\mathbf{c} \leftarrow \text{Encoder}(\mathcal{I})$ ▷ *run once, cache*
2: $\{t_0, \ldots, t_N\} \leftarrow \text{VPSchedule}(N, \beta_0, \beta_1)$
3: $\mathbf{x}_{t_0} \leftarrow \mathbf{0}$ ▷ *zero init (temp.= 0)*
4: **for** $i = 0$ **to** $N - 1$ **do**
5: $\quad \hat{\mathbf{x}}_0^{(i)} \leftarrow \text{DiT}(\mathbf{x}_{t_i}, \mathbf{c}, t_i)$ ▷ *single step*
6: $\quad$ **if** $i = 0$ **or** $p = 1$ **then**
7: $\quad\quad \mathbf{x}_{t_{i+1}} \leftarrow$ Eq. (2) ▷ *1st-order*
8: $\quad$ **else**
9: $\quad\quad \mathbf{x}_{t_{i+1}} \leftarrow$ Eq. (3) ▷ *2nd-order*
10: $\quad$ **end if**
11: $\quad \mathbf{x}_{t_{i+1}}[\text{ego}, 0] \leftarrow \mathbf{s}_{\text{cur}}$ ▷ *constrain*
12: **end for**
13: $\hat{\mathbf{x}}_0 \leftarrow \text{DiT}(\mathbf{x}_{t_N}, \mathbf{c}, t_N)$ ▷ *denoise-to-zero*
14: $\mathbf{y}_{\text{turn}} \leftarrow \text{TurnInd}(\hat{\mathbf{x}}_0, \mathbf{c})$
15: $\hat{\mathbf{x}}_0 \leftarrow \text{Denorm}(\hat{\mathbf{x}}_0)$
16: **return** $\hat{\mathbf{x}}_0, \mathbf{y}_{\text{turn}}$

The timestep schedule follows logSNR-uniform spacing [11]: given N steps, we uniformly partition the logSNR range $[\lambda_{\min}, \lambda_{\max}]$ into $N{+}1$ values and invert each to obtain continuous timesteps $t_i \in (0, 1]$. This concentrates steps in the high-noise region where the denoising trajectory changes most rapidly. After each solver update, the current-timestep slice of the trajectory tensor is overwritten with observed ego and neighbor agent states, anchoring the prediction to physical reality, as shown in Algorithm 1, line 9.

A key benefit of the modular design is encoder caching. Because the context encoder output $\mathbf{c}$ depends only on the scene context $\mathcal{I}$ and not on the denoising state $\mathbf{x}_t$, it can be computed once and reused for all $N{+}1$ DiT invocations, reducing the inference cost from T_{mono} in Eq. 4 to:

$$T_{\text{mod}} = T_{\text{enc}} + (N{+}1) \cdot T_{\text{dit}} + N \cdot T_{\text{sol}}. \quad (5)$$

The saving of $N \cdot T_{\text{enc}}$ is substantial because $T_{\text{enc}} \gg T_{\text{dit}}$ (§V-C). The model operates in a normalized coordinate frame; output trajectories are denormalized to world coordinates before being published as ROS 2 trajectory messages, ready for direct consumption by downstream planning validation and vehicle control modules within the Autoware stack.

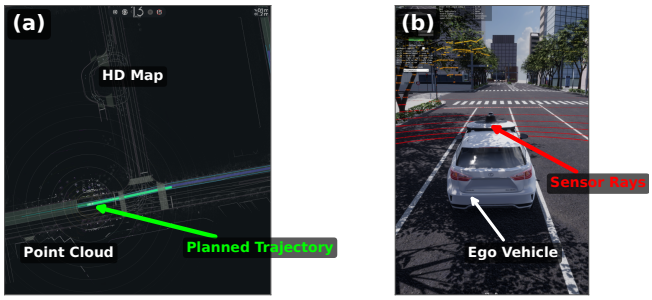

Fig. 2. Closed-loop evaluation environment. **(a)** RViz visualization showing the HD map, point cloud, and planned trajectory within the Autoware stack. **(b)** AWSIM 3D simulation at an urban intersection in the Nishishinjuku map, with the ego vehicle and sensor perception rays.

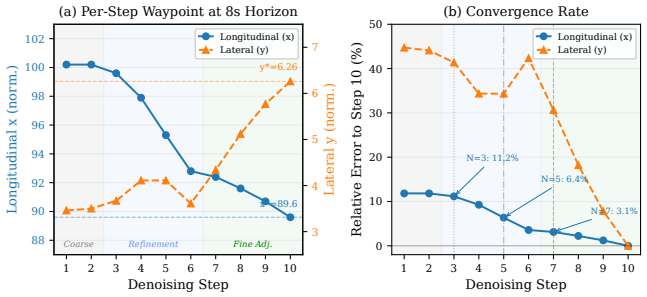

Fig. 3. Denoising progression of the 8s-horizon waypoint. **(a)** Per-step predicted position in normalized coordinates: longitudinal (blue) and lateral (orange); three phases are visible: coarse (steps 1–2), refinement (3–6), and fine adjustment (7–10). **(b)** Relative error to the converged step-10 prediction.

C. ROS 2 Integration with Autoware

The modular planner is implemented as a drop-in replacement for the original Autoware diffusion planner node. The ROS 2 node subscribes to odometry, tracked objects, vector map, route, and traffic signal topics, performs preprocessing identical to the original node, and publishes trajectory and turn indicator outputs. All solver parameters (N, p, β_0, β_1) are exposed as ROS 2 parameters, enabling runtime reconfiguration via `ros2 param set` without node restart.

V. Experiments

A. Experimental Setup

All experiments are conducted on a desktop PC with an Intel Core i9-12900KS CPU, 32 GB RAM, and an NVIDIA GeForce RTX 3090 Ti GPU, running in a Docker container with Ubuntu 22.04, ROS 2 Humble, and Autoware. The modular planner uses ONNX Runtime 1.23 with the CPU execution provider; the original monolithic planner [28] uses TensorRT with the 68 MB engine file on GPU. Both planners are benchmarked on the same machine; the CPU-based comparison isolates the architectural benefit of encoder caching from hardware acceleration effects. Closed-loop evaluation is performed in AWSIM [14], the open-source simulator integrated with Autoware [13], on an urban driving route in the Nishishinjuku map involving straight roads, intersections, and traffic participants. Unless stated otherwise, the modular planner uses $N = 10$ steps, second-order solver ($p = 2$), VP schedule with $\beta_0 = 0.1$, $\beta_1 = 20.0$, and temperature $= 0$ for deterministic sampling with zero initial noise, consistent with the original deployment. Fig. 2 shows the evaluation environment: the RViz visualization of the Autoware stack (left) and the AWSIM 3D simulation (right).

To verify that graph decomposition preserves model semantics, we compare outputs of the original monolithic model [28] and the modular pipeline on captured AWSIM driving data. For each output tensor $\mathbf{y}$, we measure the maximum element-wise absolute error:

$$\epsilon = \max_i \left| y_i^{\text{mono}} - y_i^{\text{mod}} \right|. \tag{6}$$

For individual modules such as the context encoder and a single DiT step, $\epsilon < 10^{-5}$. After 10 sequential solver steps, accumulated floating-point rounding yields $\epsilon < 10^{-4}$ for the final trajectory. Since the planner operates in a normalized coordinate frame with standard deviation $\sigma = 20$ m, the denormalization $x_{\text{phys}} = x_{\text{norm}} \cdot \sigma + \mu$ bounds the physical-space error to $\epsilon \cdot \sigma < 10^{-4} \times 20 = 2 \times 10^{-3}$ m, i.e., less than 2 mm, negligible for planning.

B. Denoising Analysis and Solver Comparison

A key capability of the modular architecture is *per-step observability*: because the C++ solver explicitly calls the DiT core at each step, we can log intermediate predictions, information that remains a black box in the monolithic deployment. Fig. 3(a) illustrates this using actual data from an AWSIM driving frame. The denoising process exhibits three distinct phases: a *coarse* phase (steps 1–2) where the prediction remains near the noise prior with minimal change, a *refinement* phase (steps 3–6) where the largest per-step changes occur as both axes converge rapidly, and a *fine adjustment* phase (steps 7–10) where residual corrections diminish. The lateral position refines non-monotonically, indicating that curvature adjustment lags behind the longitudinal structure. Fig. 3(b) quantifies the convergence rate: the longitudinal relative error drops to 6.4% by step 5 and below 3.1% by step 7, while the lateral channel converges more slowly due to its non-monotonic refinement. This diagnostic capability, combined with runtime-configurable N, p, and (β_0, β_1), enables systematic solver analysis without model recompilation.

Since the refinement phase concentrates most of the prediction change, reducing step count primarily truncates the fine adjustment phase. At N=7, with 27% fewer DiT calls, the mean 8s-horizon waypoint deviation is only 0.87 m, well within the replanning margin of a 10 Hz planner that executes only the first 1–2 s of each trajectory. Even at N=5 with 45% reduction, the 1.71 m deviation may be acceptable for latency-critical scenarios, supporting the feasibility of

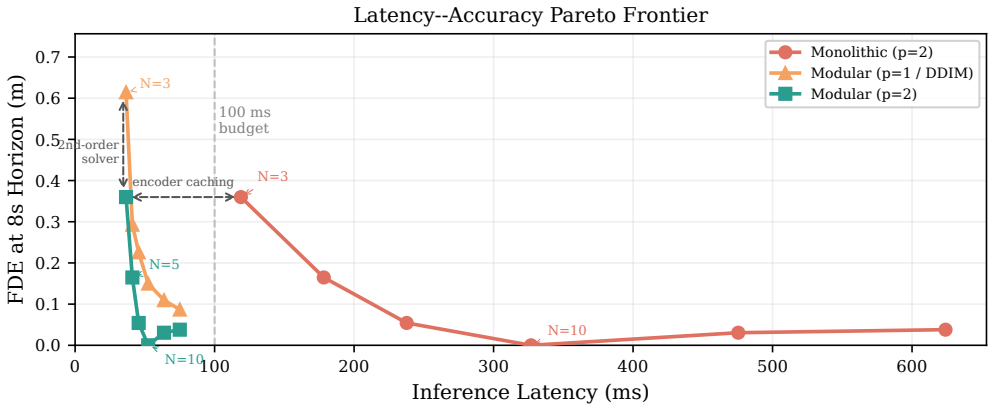

Fig. 4. Latency–accuracy Pareto frontier. Horizontal gap: encoder caching benefit; vertical gap: solver order benefit. Dashed line: 100 ms planning budget.

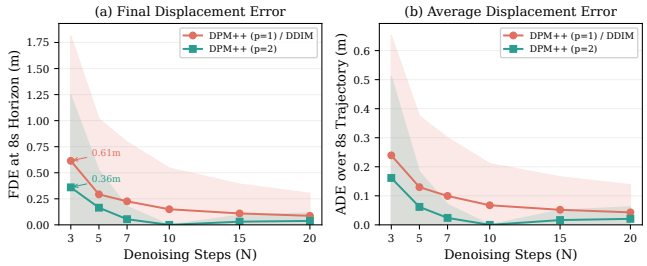

Fig. 5. Solver comparison. **(a)** FDE and **(b)** ADE vs. $N{=}10$, $p{=}2$ reference. Shaded: $\pm 1\sigma$.

anytime planning [9] where step count adapts to available compute budget.

To further demonstrate the pluggable solver interface, we conduct an offline benchmark: for each of 50 evenly sampled AWSIM frames, we replay the captured input tensors through the DiT core with three solver configurations (DPM-Solver++ first-order ($p{=}1$), second-order ($p{=}2$), and DDIM [15]) across $N \in \{3, 5, 7, 10, 15, 20\}$. Each configuration uses its own logSNR-uniform timestep schedule optimized for the given N. Fig. 5 reports Final Displacement Error (FDE) and Average Displacement Error (ADE) relative to the $N{=}10$, $p{=}2$ reference. DPM-Solver++ first-order produces outputs identical to DDIM across all configurations, empirically confirming the known theoretical equivalence for data-prediction models [11]. The second-order solver provides substantial gains. At $N{=}3$, FDE drops by 41% relative to first-order, from 0.61 m to 0.36 m, and the improvement is consistent across all N, with $p{=}2$ achieving the same accuracy as $p{=}1$ at roughly half the steps. Proper timestep scheduling is equally important: comparing with the early-stopping analysis above, a dedicated $N{=}3$ schedule yields 0.36 m FDE vs. 2.53 m when truncating a 10-step schedule, a 7× improvement.

Because the modular architecture exposes the solver as a configurable component, researchers can directly evaluate novel scheduling strategies, step-adaptive policies, and alternative solvers on driving data captured from a production AD stack, rather than relying on dataset replay in standalone simulators.

TABLE II

INFERENCE LATENCY BREAKDOWN.

Component	**Time**	**Mono. calls**	**Mod. calls**
Context Encoder	27.5 ms	$N{+}1$	1
DiT Core† (per step)	2.3 ms	$N{+}1$	$N{+}1$
C++ Solver overhead	0.01 ms/step	—	N
Total ($N{=}10$)		**328 ms**	**53 ms**

†$N{+}1$: N solver steps + 1 final denoise-to-zero (Alg. 1, line 10).

C. System Performance and Encoder Caching

While the previous sections analyzed algorithmic convergence, this section evaluates the system-level metric that matters for autonomous driving: *inference latency*.

Table II decomposes the inference latency, measured via ONNX Runtime on CPU over 100 runs with the median reported. The context encoder dominates the per-step cost at $T_{\text{enc}} = 27.5$ ms. In the monolithic architecture, this encoder is redundantly re-executed at every denoising step due to the fused graph structure, a 90.9% computational waste. Our modular architecture runs it *once* and caches the output, reducing the marginal cost of each additional step to just $T_{\text{dit}} = 2.3$ ms. Note that the monolithic 328 ms in Table II is the equivalent CPU cost computed from the same per-component latencies; the actual monolithic planner runs on GPU via TensorRT, so this value isolates the architectural difference rather than the hardware difference.

Fig. 4 plots inference latency against FDE for three configurations; the ideal operating point lies in the lower-left corner, where both latency and prediction error are minimized. The horizontal arrow in the figure illustrates the encoder caching benefit: at $N{=}3$ with the same second-order solver, switching from monolithic to modular reduces latency

from 119 ms to 37 ms, a **3.2**× speedup at identical FDE. The vertical arrow shows the solver order benefit: at the same latency, second-order solving at $N{=}3$ matches the accuracy of first-order at $N{=}5$, effectively halving the required steps. Notably, *all* modular configurations fall within the 100 ms planning budget, while *no* monolithic configuration does; even the fastest monolithic point already exceeds 119 ms. This means the planner can dynamically select step count based on the remaining time in each 100 ms ROS 2 cycle: when compute is abundant, use $N{=}10$ for maximum accuracy; when a previous module overruns, fall back to $N{=}3$ and still meet the deadline with acceptable quality.

We validate the system in AWSIM closed-loop simulation, running the full Autoware stack with our node as a drop-in replacement for the original monolithic planner [28]. Over 600+ planning cycles on the Nishishinjuku urban route, the ego vehicle completes the route without collisions, maintaining a consistent $\sim$10 Hz output rate at 53 ms inference plus preprocessing.

VI. Discussion and Conclusion

We presented an open-source modular benchmark that decomposes a monolithic ONNX diffusion planner into independently executable modules within the Autoware production AD stack and AWSIM closed-loop simulator. The modular design *decouples* architectural and algorithmic optimization: encoder caching alone yields a 3.2× latency reduction, while upgrading to the second-order solver reduces FDE by 41% at $N{=}3$. These two improvements are orthogonal and compound while maintaining numerical equivalence.

The decomposition relies only on a repeated substructure conditioned on a cacheable context, a pattern shared by recent diffusion planners [1]–[4] and scene generators [29], making the approach readily transferable. To our knowledge, no existing open-source framework provides pluggable solver evaluation under true closed-loop conditions in a ROS 2-based AD stack.

Our current evaluation covers a single urban scenario in AWSIM. Broader scenario coverage, systematic GPU latency profiling, and analysis of inter-module transfer overhead at very low step counts are left to future work. The complete codebase will be released as open-source to support reproducible research in closed-loop diffusion planning.

References

[1] Y. Zheng, R. Liang, K. Zheng, J. Zheng, L. Mao, J. Li, W. Gu, R. Ai, S. E. Li, X. Zhan, and J. Liu, "Diffusion-based planning for autonomous driving with flexible guidance," in *The Thirteenth International Conference on Learning Representations*, 26 Jan. 2025.

[2] B. Liao, S. Chen, H. Yin, B. Jiang, C. Wang, S. Yan, X. Zhang, X. Li, Y. Zhang, Q. Zhang, and X. Wang, "DiffusionDrive: Truncated diffusion model for end-to-end autonomous driving," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. IEEE, 22 Nov. 2025. [Online]. Available: https://openaccess.thecvf.com/content/CVPR2025/papers/Liao_DiffusionDrive_Truncated_Diffusion_Model_for_End-to-End_Autonomous_Driving_CVPR_2025_paper.pdf

[3] S. Liu, W. Chen, W. Li, Z. Wang, L. Yang, J. Huang, Y. Zhang, Z. Huang, Z. Cheng, and H. Yang, "BridgeDrive: Diffusion Bridge Policy for Closed-Loop Trajectory Planning in Autonomous Driving," in *The Fourteenth International Conference on Learning Representations*, 8 Oct. 2026. [Online]. Available: https://openreview.net/forum?id=dJKhjK4zpp

[4] J. Zhang, W. Xia, Z. Zhou, Y. Gong, and J. Mei, "LAP: Fast LAtent diffusion planner with fine-grained feature distillation for autonomous driving," *arXiv [cs.RO]*, 2 Dec. 2025. [Online]. Available: http://arxiv.org/abs/2512.00470

[5] N. Karnchanachari, D. Geromichalos, K. S. Tan, N. Li, C. Eriksen, S. Yaghoubi, N. Mehdipour, G. Bernasconi, W. K. Fong, Y. Guo, and H. Caesar, "Towards learning-based planning: The nuPlan benchmark for real-world autonomous driving," in *2024 IEEE International Conference on Robotics and Automation (ICRA)*, vol. 56. IEEE, 13 May 2024, pp. 629–636. [Online]. Available: http://dx.doi.org/10.1109/ICRA57147.2024.10610077

[6] D. Dauner, M. Hallgarten, A. Geiger, and K. Chitta, "Parting with Misconceptions about Learning-based Vehicle Motion Planning," in *Conference on Robot Learning*. PMLR, 2 Dec. 2023, pp. 1268–1281. [Online]. Available: https://proceedings.mlr.press/v229/dauner23a.html

[7] T. Betz, M. Schmeller, H. Teper, and J. Betz, "How fast is my software? Latency evaluation for a ROS 2 autonomous driving software," in *2023 IEEE Intelligent Vehicles Symposium (IV)*. IEEE, 4 Jun. 2023, pp. 1–6. [Online]. Available: http://dx.doi.org/10.1109/IV55152.2023.10186585

[8] T. Betz, H. Teper, D. Ebner, M. Leitenstern, S. Sagmeister, M. Weinmann, J.-J. Chen, and M. Lienkamp, "End-to-end latency optimization for containerized ROS 2 autonomous driving software," *IEEE Access*, vol. 13, pp. 112 654–112 672, 2025. [Online]. Available: http://dx.doi.org/10.1109/ACCESS.2025.3582868

[9] M. Likhachev, G. J. Gordon, and S. Thrun, "ARA*: Anytime A* with provable bounds on sub-optimality," in *Advances in Neural Information Processing Systems*, vol. 16, 9 Dec. 2003, pp. 767–774. [Online]. Available: https://proceedings.neurips.cc/paper/2003/hash/ee8fe9093fbbb687bef15a38facc44d2-Abstract.html

[10] NVIDIA, "ONNX GraphSurgeon," https://github.com/NVIDIA/TensorRT/tree/main/tools/onnx-graphsurgeon, 2024.

[11] C. Lu, Y. Zhou, F. Bao, J. Chen, C. Li, and J. Zhu, "DPM-solver++: Fast solver for guided sampling of diffusion probabilistic models," *arXiv [cs.LG]*, 2 Nov. 2022. [Online]. Available: http://arxiv.org/abs/2211.01095

[12] Y. Song, J. Sohl-Dickstein, D. P. Kingma, A. Kumar, S. Ermon, and B. Poole, "Score-Based Generative Modeling through Stochastic Differential Equations," in *International Conference on Learning Representations*, 2 Oct. 2020. [Online]. Available: https://openreview.net/forum?id=PxTIG12RRHS&utm_campaign=NLP%20News&utm_medium=email&utm_source=Revue%20newsletter

[13] The Autoware Foundation, "Autoware: Introducing Open-Source End-to-End Autonomous Driving," The Autoware Foundation, Tech. Rep., 2025, technical Report. Available at https://autoware.org/wp-content/uploads/2025/10/Autoware-E2E-Report_Release.pdf.

[14] TIER IV, Inc., "AWSIM: Open-source autonomous driving simulator for Autoware," https://github.com/tier4/AWSIM, 2024.

[15] J. Song, C. Meng, and S. Ermon, "Denoising Diffusion Implicit Models," in *International Conference on Learning Representations*, 2 Oct. 2021. [Online]. Available: https://openreview.net/forum?id=St1giarCHLP

[16] J. Ho, A. Jain, and P. Abbeel, "Denoising Diffusion Probabilistic Models," in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 6840–6851. [Online]. Available: https://proceedings.neurips.cc/paper/2020/hash/4c5bcfec8584af0d967f1ab10179ca4b-Abstract.html

[17] M. Janner, Y. Du, J. Tenenbaum, and S. Levine, "Planning with Diffusion for Flexible Behavior Synthesis," in *International Conference on Machine Learning*. PMLR, 28 Jun. 2022, pp. 9902–9915. [Online]. Available: https://proceedings.mlr.press/v162/janner22a.html

[18] C. Chi, S. Feng, Y. Du, Z. Xu, E. Cousineau, B. Burchfiel, and S. Song, "Diffusion policy: Visuomotor policy learning via action diffusion," in *Robotics: Science and Systems XIX*. Robotics:

Science and Systems Foundation, 10 Jul. 2023. [Online]. Available: http://dx.doi.org/10.15607/rss.2023.xix.026

[19] W. Peebles and S. Xie, "Scalable diffusion models with transformers," in *Proceedings of the IEEE/CVF international conference on computer vision*, 2023, pp. 4195–4205. [Online]. Available: http://openaccess.thecvf.com/content/ICCV2023/html/Peebles_Scalable_Diffusion_Models_with_Transformers_ICCV_2023_paper.html

[20] B. Yang, H. Su, N. Gkanatsios, T.-W. Ke, A. Jain, J. Schneider, and K. Fragkiadaki, "Diffusion-ES: Gradient-free planning with diffusion for autonomous driving and zero-shot instruction following," *arXiv [cs.LG]*, 9 Feb. 2024. [Online]. Available: http://arxiv.org/abs/2402.06559

[21] T. Xia, Y. Li, L. Zhou, J. Yao, K. Xiong, H. Sun, B. Wang, K. Ma, G. Chen, H. Ye, W. Liu, and X. Wang, "DriveLaW:Unifying planning and video generation in a latent driving world," *arXiv [cs.CV]*, 31 Dec. 2025. [Online]. Available: http://arxiv.org/abs/2512.23421

[22] Baidu, "Apollo: Open source autonomous driving platform," https://github.com/ApolloAuto/apollo, 2024.

[23] Microsoft, "ONNX Runtime: cross-platform, high performance ML inferencing and training accelerator," https://onnxruntime.ai/, 2019.

[24] NVIDIA, "NVIDIA TensorRT," https://developer.nvidia.com/tensorrt, 2024.

[25] X. Guo, Q. Jiang, Y. Shen, A. D. Pimentel, and T. Stefanov, "EASTER: Learning to split transformers at the edge robustly," *IEEE Trans. Comput.-aided Des. Integr. Circuits Syst.*, vol. 43, no. 11, pp. 3626–3637, Nov. 2024. [Online]. Available: http://dx.doi.org/10.1109/TCAD.2024.3438995

[26] R. C. Nair, M. Rao, and Muralidhara, "SecureFrameNet:A rich computing capable secure framework for deploying neural network models on edge protected system," in *The Third International Conference on Artificial Intelligence and Machine Learning Systems*. New York, NY, USA: ACM, 25 Oct. 2023, pp. 1–13. [Online]. Available: http://dx.doi.org/10.1145/3639856.3639864

[27] P. Jajal, W. Jiang, A. Tewari, E. Kocinare, J. Woo, A. Sarraf, Y.-H. Lu, G. K. Thiruvathukal, and J. C. Davis, "Interoperability in deep learning: A user survey and failure analysis of ONNX model converters," in *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*, vol. 18. New York, NY, USA: ACM, 11 Sep. 2024, pp. 1466–1478. [Online]. Available: http://dx.doi.org/10.1145/3650212.3680374

[28] TIER IV, Inc., "Autoware Diffusion Planner," https://github.com/autowarefoundation/autoware_universe/tree/main/planning/autoware_diffusion_planner, 2025.

[29] C. M. Jiang, Y. Bai, A. Cornman, C. Davis, X. Huang, H. Jeon, S. Kulshrestha, J. Lambert, S. Li, X. Zhou, C. Fuertes, C. Yuan, M. Tan, Y. Zhou, and D. Anguelov, "SceneDiffuser: Efficient and controllable driving simulation initialization and rollout," in *Advances in Neural Information Processing Systems 37*, A. Globerson, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. Tomczak, and C. Zhang, Eds., vol. 37. San Diego, California, USA: Neural Information Processing Systems Foundation, Inc. (NeurIPS), 2024, pp. 55 729–55 760. [Online]. Available: http://dx.doi.org/10.52202/079017-1771